

OpenAI Chat Completions API 기본

이 튜토리얼은 OpenAI의 Chat Completions API를 활용하여 챗봇이나 AI 기능을 개발하는 방법을 단계별로 설명합니다. 특히 OpenAI의 최신 언어 모델 중 하나인 GPT-4o-mini/gpt-4.1-nano를 사용하여 예제를 진행할 것입니다. 각 섹션에는 개념 설명과 함께 실행 가능한 파이썬 코드 예제가 포함되어 있습니다.

주요 학습 내용:

1. OpenAI API 소개 및 환경 설정: OpenAI API 개요, API 키 발급 및 보안 설정, 파이썬 클라이언트 설치 및 인스턴스 생성 방법
2. 기본적인 Chat Completions API 사용법: 간단한 대화형 텍스트 생성 요청과 응답 처리, 프롬프트 엔지니어링 기초
3. 스트리밍 응답: 대화 응답을 스트리밍 방식으로 받아 실시간 처리하는 방법
4. 시스템 메시지 활용: 시스템 역할 메시지를 사용하여 AI의 응답 스타일이나 행동을 조정하는 방법
5. 고급 활용법: 토큰 최적화와 비용 절감 전략, OpenAI API 에러 처리 및 예외Handling
6. 실전 프로젝트 예제: 간단한 챗봇 구현 및 외부 데이터/API와 연동하여 데이터 분석 기능을 결합한 사례

1. OpenAI API 소개 및 환경 설정

먼저 OpenAI API와 Chat Completions에 대해 간략히 알아보고, API를 사용하기 위한 환경을 설정해보겠습니다.

OpenAI API 개요

OpenAI API는 GPT 계열의 대규모 언어 모델을 인터넷을 통해 사용할 수 있도록 제공하는 서비스입니다. Chat Completions API는 챗봇과 유사한 대화형 상호작용을 할 수 있는 엔드포인트로, 역할(role)이 부여된 메시지 목록을 입력하면 모델이 다음 대화 내용을 생성합니다. GPT-4o는 텍스트

와 이미지 입력을 모두 처리하며 최대 128k 토큰의 긴 문맥을 다룰 수 있습니다. GPT-4o와 경량화 모델인 GPT-4o-mini 등이 제공되며, 요구 사항에 따라 적절한 모델을 선택할 수 있습니다 (GPT-4o-mini는 비용 효율이 높음)

API 키 발급 및 보안 설정

OpenAI API를 사용하려면 먼저 OpenAI 계정에서 API 키를 발급받아야 합니다. OpenAI 웹사이트의 API Keys 페이지에서 새로운 비밀 키를 생성할 수 있습니다. 발급받은 API 키는 비밀로 관리해야 하며, 소스 코드나 공개 저장소에 노출되지 않도록 주의해야 합니다. 가장 좋은 방법은 API 키를 코드에 하드코딩하지 않고, 환경 변수나 별도의 설정 파일에 저장하는 것입니다. 이 튜토리얼에서는 .env 파일에 키를 저장하고 파이썬에서 이를 불러오는 방식을 사용합니다. 이를 위해 Python용 패키지 `**python-dotenv**`를 활용하겠습니다.

- .env 파일에 키 저장: 프로젝트 디렉터리에 .env 파일을 만들고 아래와 같이 API 키를 저장합니다 (따옴표 없이).

```
...  
  
OPENAI_API_KEY=발급받은-API키-값  
  
...
```

- python-dotenv 사용: 파이썬 코드에서 python-dotenv를 이용해 .env 파일의 환경 변수를 불러올 수 있습니다.

위 코드로 client 객체가 생성되었습니다. 이제 이 client를 통해 OpenAI Chat Completions API를 호출할 수 있습니다. 다음 섹션부터는 실제로 Chat Completions API를 호출하여 다양한 기능을 실습해보겠습니다.

2. 기본적인 Chat Completions API 사용법

이 섹션에서는 Chat Completions API를 사용하여 가장 기본적인 대화 생성 작업을 수행해봅니다.

간단한 텍스트 생성 요청

Chat Completions 엔드포인트는 메시지 목록을 입력으로 받아 다음에 이어질 메시지를 생성합니다. 각 메시지는 role과 content 필드로 구성되어 있으며, 일반적으로 ****user** (사용자 메시지), **assistant** (모델의 응답 메시지), **system** (시스템 지시 메시지)** 세 가지 역할을 사용합니다. 가장 간단한 예제로, 사용자 역할의 메시지 하나를 모델에 보내고 응답을 받아보겠습니다. 모델은 GPT-4o를 사용합니다.

3. 스트리밍 응답 (Streaming)

기본적으로 OpenAI API는 요청에 대한 완료된 답변을 한꺼번에 반환합니다. 그러나 긴 답변의 경우 스트리밍을 사용하면 마치 타이핑을 하듯이 토큰 단위로 차례로 응답을 받을 수 있습니다. 스트리밍을 활용하면 사용자에게 실시간으로 응답을 표시하거나, 매우 긴 응답을 부분 부분 처리할 수 있습니다.

스트리밍이 필요한 경우

- 실시간 피드백: 사용자 경험을 개선하기 위해 답변 생성을 기다리는 동안 실시간으로 텍스트를 보여줄 때.
- 긴 응답 처리: 응답이 길어서 한꺼번에 받으면 메모리 사용이 많을 때, 토큰이 도착하는 대로 처리 가능.
- 중간 작업 가능: 응답을 받는 도중에도 다른 이벤트를 처리하거나 UI 업데이트를 할 수 있음.

스트리밍 사용 방법

OpenAI 파이썬 라이브러리에서 스트리밍을 사용하려면 요청 시 `stream=True` 옵션을 주면 됩니다. 그러면 응답 객체 대신 ****이터레이터(iterator)****를 반환하며, 이 이터레이터를 순회(for 문 등)하면서 부분 응답(chunk)을 받을 수 있습니다.

다음은 스트리밍 응답을 처리하는 코드 예제입니다:

위 코드를 실행하면 `response_stream`은 응답 스트림 객체가 되고, `for` 루프에서 순차적으로 응답 조각을 받아옵니다. 각 `chunk`는 `choices[0].delta`에 현재 추가 생성된 텍스트 조각을 담고 있습니다 (완전한 메시지가 아니라 추가된 부분만을 담음). 이를 이어붙여 화면에 출력하면 모델이 답변을 조금씩 생성해가는 과정을 실시간으로 볼 수 있습니다. 예를 들어, 모델이 "안녕하세요, 만나서 반갑습니다."라는 문장을 생성한다면, 스트리밍 출력은 사람이 타이핑하듯 안, 안녕, 안녕하세요, ... 차례로 출력될 것입니다. 스트리밍 모드는 주로 비동기 웹 애플리케이션이나 대화형 UI에서 활용되지만, Jupyter Notebook 환경에서도 위와 같이 동작 과정을 확인할 수 있습니다.

4. 시스템 메시지 활용

****시스템 메시지(system role message)****는 모델에게 전체 대화의 맥락이나 규칙을 알려주는 역할을 합니다. 시스템 메시지를 활용하면 AI의 말투, 행동 방식, 응답 형식 등을 조정할 수 있습니다. 시스템 메시지는 대화의 첫 번째 메시지로 넣는 경우가 많으며, 사용자에게는 보이지 않지만 모델에게는 강한 지침으로 작용합니다.

시스템 메시지의 역할

- 행동 지침: 모델이 따라야 할 규칙이나 목표를 제시 (예: "반말로 대답하지 마세요", "모든 응답에 이모티콘 하나를 포함하세요").
- 역할 부여: 모델에게 특정 인격이나 역할을 부여 (예: "너는 역사 전문가야", "너는 사용자를 돕는 비서야").
- 컨텍스트 설정: 대화 주제나 맥락을 사전에 설정 (예: "이 대화는 의료 상담입니다", "사용자는 프로그래밍 도움을 요청할 것입니다").

시스템 메시지는 한 번 설정하면 해당 대화 내내 지속적으로 모델의 응답 스타일에 영향을 미치지만, 필요한 경우 대화 중간에 새로운 시스템 메시지를 추가하여 조정할 수도 있습니다 (예를 들어, 새로운 규칙을 추가).

시스템 메시지 사용 예제

시스템 메시지를 사용하여 모델의 말투를 바꿔보겠습니다. 모델에게 "해적처럼 말하는 코딩 도우미"라는 캐릭터를 부여한 후, 사용자의 질문에 답하게 해보겠습니다.

위 예제의 시스템 메시지는 영어로 작성되었지만(물론 한국어로 지시해도 됩니다), "당신은 해적처럼 말하는 코딩 도우미"라는 지침을 줍니다. 그 다음 사용자 질문은 일반적으로 "Python에서 객체가 특정 클래스의 인스턴스인지 어떻게 확인하나요?"라는 내용입니다. 시스템 메시지 덕분에, 모델의 답변은 아마도 해적 말투로 나올 것입니다.

이처럼 동일한 질문이라도 시스템 메시지를 통해 모델의 답변 스타일이나 관점을 크게 바꿀 수 있습니다. 필요에 따라 시스템 메시지를 활용하여 프로젝트의 톤앤매너에 맞는 응답을 얻도록 조정하세요.

> 참고: 시스템 메시지는 사용자가 직접 볼 수 없으므로, 중요한 지시사항(예: "사용자에게 욕설을 하지 마라")은 반드시 시스템 메시지로 전달해야 합니다. 모델은 사용자 메시지의 내용보다 시스템 메시지의 지시에 우선순위를 두도록 설계되어 있습니다.

5. 고급 활용법

이 섹션에서는 Chat Completions API를 보다 효율적으로 사용하기 위한 고급 기법들을 다룹니다. 토큰 사용을 최적화하여 비용을 절감하는 방법과, API 호출 시 발생할 수 있는 오류를 처리하는 방법을 설명합니다.

토큰 최적화 및 비용 절감

OpenAI API 비용은 사용한 토큰(token) 수에 비례하여 청구됩니다. 따라서 동일한 작업을 하더라도 토큰을 적게 사용하면 비용이 줄어들고, 응답 속도도 빨라집니다. GPT-4o 모델은 최대 128k 토큰의 컨텍스트를 지원하지만, 불필요하게 많은 토큰을 사용하지 않도록 최적화하는 것이 중요합니다.

토큰 최적화를 위한 팁:

- 짧고 명확한 프롬프트: 시스템 메시지와 사용자 메시지를 불필요하게 장황하게 쓰지 않고 간결하게 작성합니다. 예를 들어 동일한 지시라도 "간결하게 답변해주세요."는 "부디 당신의 답변을 최대한 간략하게 제공해 주셨으면 합니다."보다 적은 토큰으로 같은 의미를 전달합니다.
- 대화 내역 관리: 이전 대화 기록을 얼마나 포함시킬지 결정해야 합니다. 모든 이전 메시지를 매

번 보낼 필요는 없습니다. 중요한 맥락만 남기고 요약하거나 일부 생략하여 토큰을 줄입니다.

- 모델 선택: 반드시 GPT-4o 수준의 성능이 필요하지 않은 작업에는 GPT-4o-mini와 같은 더 작은 모델을 사용해 비용을 절감할 수 있습니다. (GPT-4o-mini는 GPT-4o보다 비용이 훨씬 저렴하여 일상적인 작업에 적합합니다.)

- max_tokens 파라미터 활용: 응답의 최대 길이를 설정하여 너무 긴 답변이 나오지 않도록 제어합니다. 예를 들어 요약 생성 등의 작업에서는 max_tokens를 짧게 설정해 모델이 알아서 간결한 답을 내놓게 유도할 수 있습니다.

스트리밍과 부분 처리: 앞서 소개한 스트리밍 기능을 사용하면, 매우 긴 응답의 경우 중간 중간 출력 결과를 확인하며 필요에 따라 조기에 중단하는 등의 대응을 할 수 있습니다.

추가로, OpenAI는 Batch API 등을 통해 다수의 요청을 한 번에 보내 비용을 절약하는 방법을 제공하기도 합니다. 다만 이 튜토리얼의 범위를 벗어나므로 자세한 내용은 OpenAI 공식 문서를 참고하세요.

토큰 최적화의 효과를 확인하고 싶다면, 응답 객체의 usage 정보를 출력해볼 수 있습니다. response.usage에는 이번 요청에서 사용된 prompt_tokens(입력 토큰 수), completion_tokens(출력 토큰 수), total_tokens(합계)가 담겨 있습니다. 예를 들어:

이런 정보를 토대로 모델이 과도하게 긴 답변을 내놓지는 않았는지 모니터링하고, 프롬프트를 조정하는 피드백 loop을 거치면 점점 효율적으로 API를 활용할 수 있습니다.

에러 핸들링 및 예외 처리

OpenAI API를 사용하는 애플리케이션을 개발할 때는 각종 오류 상황을 대비해야 합니다. 주로 발생할 수 있는 예외 상황과 대처 방안은 다음과 같습니다:

- 네트워크 오류 또는 타임아웃: 인터넷 연결 문제나 일시적인 서버 응답 지연으로 요청이 실패할 수 있습니다. 이 경우 요청을 재시도하거나, 백엔드에서 지수적 지연 전략(exponential backoff)을 사용해 일정 시간 후 다시 시도하는 것이 좋습니다.

- 레이트 리미트 (Rate Limit) 초과: OpenAI API는 일정 기간당 요청 허용량을 초과하면 RateLimitError를 발생시킵니다. 이 경우 일정 시간 대기 후 재시도하거나, 요청 빈도를 낮추는 조정이 필요합니다.

- 유효하지 않은 요청: 모델 이름 오타, 매개변수 형식 오류 등으로 `InvalidRequestError`가 발생할 수 있습니다. 이런 오류는 API 호출 전에 코드에서 철저한 검증을 통해 예방하는 것이 좋습니다.(Dale 쓸 때 이미지 처리시 `InvalidRequestError`생길수 있음)

- API 키 오류: 잘못된 API 키나 권한 문제로 인증 오류(`AuthenticationError`)가 발생할 수 있으므로, API 키가 정확하고 유효한지 확인해야 합니다.

파이썬 라이브러리를 사용할 때 이러한 오류들은 `openai.error` 모듈 내 예외 클래스로 나타납니다. 일반적인 최상위 예외는 `openai.error.OpenAIError`이며, 모든 OpenAI 관련 예외의 부모 클래스입니다. 간단한 예외 처리 예제를 보겠습니다:

위 코드에서 `timeout=5`는 응답이 5초 안에 없으면 `OpenAIError`를 발생시키도록 한 것으로, 강제로 타임아웃 상황을 연출하기 위한 예시입니다. `RateLimitError`는 별도로 캐치하여 사용자에게 요청 제한 메시지를 보여주고, 그 외 모든 OpenAI 오류는 일반적으로 메시지(e)를 출력하도록 했습니다. 실제 애플리케이션에서는 오류 종류에 따라 로깅을 남기고, 필요하면 재시도 로직을 넣는 등 더 정교한 대응을 구현할 수 있습니다.

마지막으로, 예상하지 못한 예외 상황(예: JSON 디코딩 오류나 타입 오류 등)이 발생할 수 있으므로, API 호출 코드 주위에는 일반 예외 처리도 넣어서 프로그램이 갑자기 중단되지 않도록 만드는 것이 좋습니다.

6. 실전 프로젝트 예제

마지막으로, 앞서 배운 내용을 종합하여 실제 응용 사례로 여러 번의 대화가 오가는 챗봇 구현을 간단히 살펴보겠습니다.

간단한 대화형 챗봇 구현

OpenAI Chat Completions API를 사용하면 비교적 적은 코드로 대화형 챗봇을 만들 수 있습니다. 여기서는 콘솔에서 사용자의 입력을 받아 모델의 응답을 출력하는 간단한 챗봇을 구현해봅니다. 이 챗봇은 이전 대화 맥락을 기억하여 연속적인 대화를 주고받을 수 있습니다.

위 코드는 while 루프를 돌면서 사용자 입력을 받습니다. "종료"라고 입력하면 루프를 빠져나와 챗봇이 종료됩니다. 각 반복에서 사용자의 입력을 chat_history에 추가한 후, 해당 chat_history를 그대로 모델에게 보내 응답을 받습니다. 응답을 출력하고, 다시 chat_history에 추가하여 맥락을 유지합니다. 시스템 메시지로 초반에 상담원으로서의 태도를 지정했기 때문에, AI는 공손하고 상세한 답변을 지속적으로 생성할 것입니다.

이처럼 간단한 구조만으로도 사용자와 지속적인 맥락을 가진 대화를 주고받는 챗봇을 만들 수 있습니다. 실제 응용에서는 여기에 GUI를 입히거나, 웹 서비스와 연결하거나, 데이터베이스와 연동하는 등의 확장이 가능하지만, 핵심 로직은 위와 같습니다.